The first thing you may notice is the use of virtual nodes. Can't you assign a hash code directly to a routee and forget about virtual nodes? Well, virtual nodes are the secret sauce of constant hashing. They are used for two reasons:

- To give a greater chance of distributing all messages evenly across routes
- To allow for the removal and addition of routes

The number of virtual nodes served by a routee must be configured when using a consistent hashing strategy by specifying the factor multiplied by the number of routees, as in the definition of `withConsistentHashingRouting` in the snippet after next.

Besides using a `ServiceKey` for a group router, you also need to provide the hashing function: the mapping from message to message key. The following snippet has both, along with the protocol for the aggregator:

```
object Aggregator {

  sealed trait Command {
    def id: String                                                          ❶
  }
  final case class Obfuscated(id: String, content: String) extends Command
  final case class Enriched(id: String, metadata: String) extends Command

  val serviceKey = ServiceKey[Aggregator.Command]("agg-key")                ❷

  def mapping(command: Command) = command.id                                ❸
...
}
```

❶ ID that allows the different parts of the same message to be forwarded to the same aggregator

❷ Service key to allow a group router to find registered aggregators

❸ Function from message to message key